

IA Moderna

Unidad 4 — Redes neuronales, visión por computador, secuencias, lenguaje y transformers.

Contenido

01 Fundamentos de redes neuronales

02 Visión por computador (clásica)

03 Redes neuronales convolucionales (CNN)

04 Modelos secuenciales (RNN, LSTM, GRU)

05 Procesamiento de lenguaje natural (PLN)

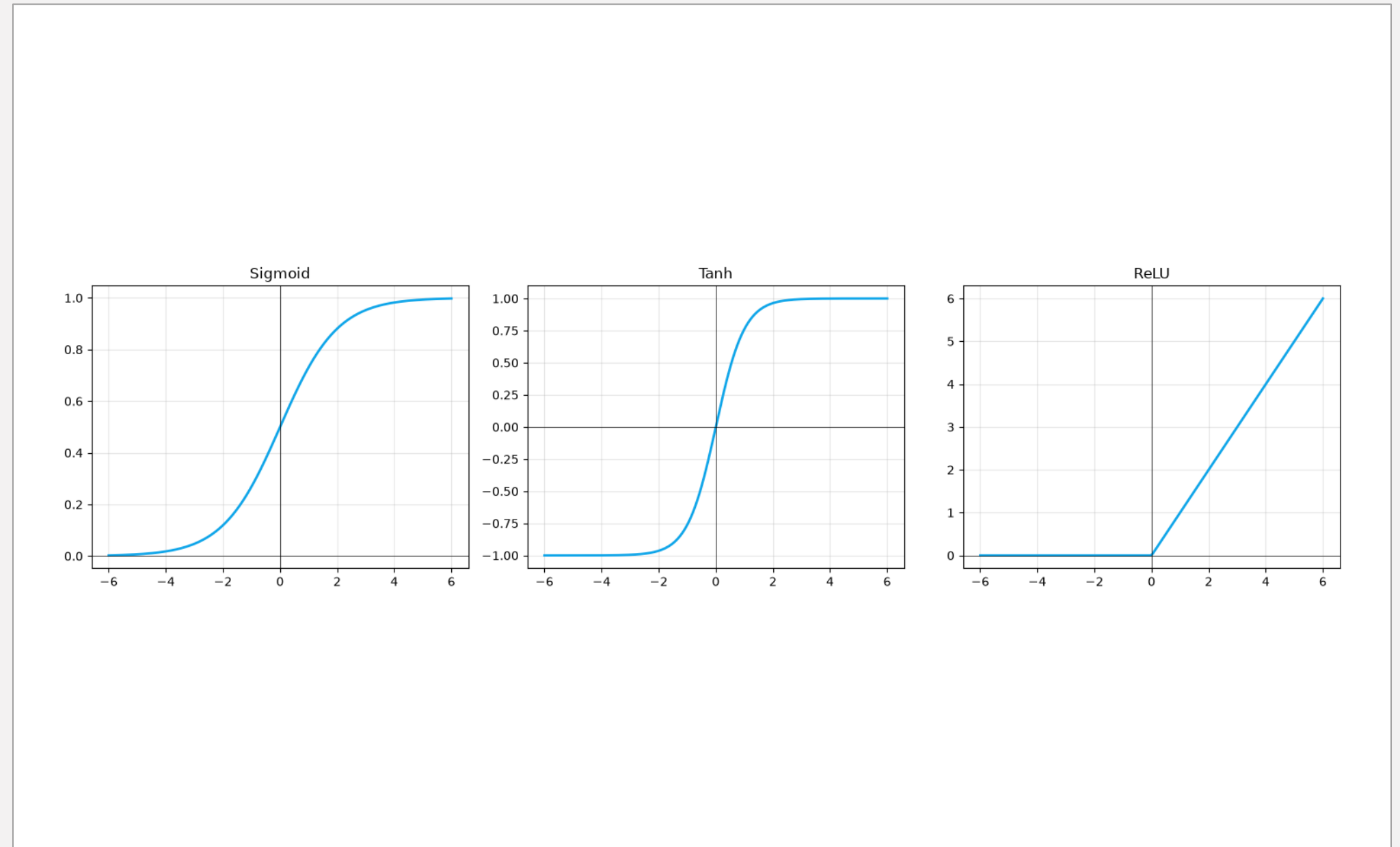
06 Transformers

01

Fundamentos de Redes Neuronales

La neurona artificial y sus funciones de activación

- Una neurona combina sus entradas linealmente (pesos + sesgo) y aplica una función de activación no lineal
- Sin activación no lineal, apilar capas seguiría siendo equivalente a un único modelo lineal
- Sigmoid y Tanh saturan en los extremos; ReLU es la más usada en capas ocultas por evitar ese problema para valores positivos

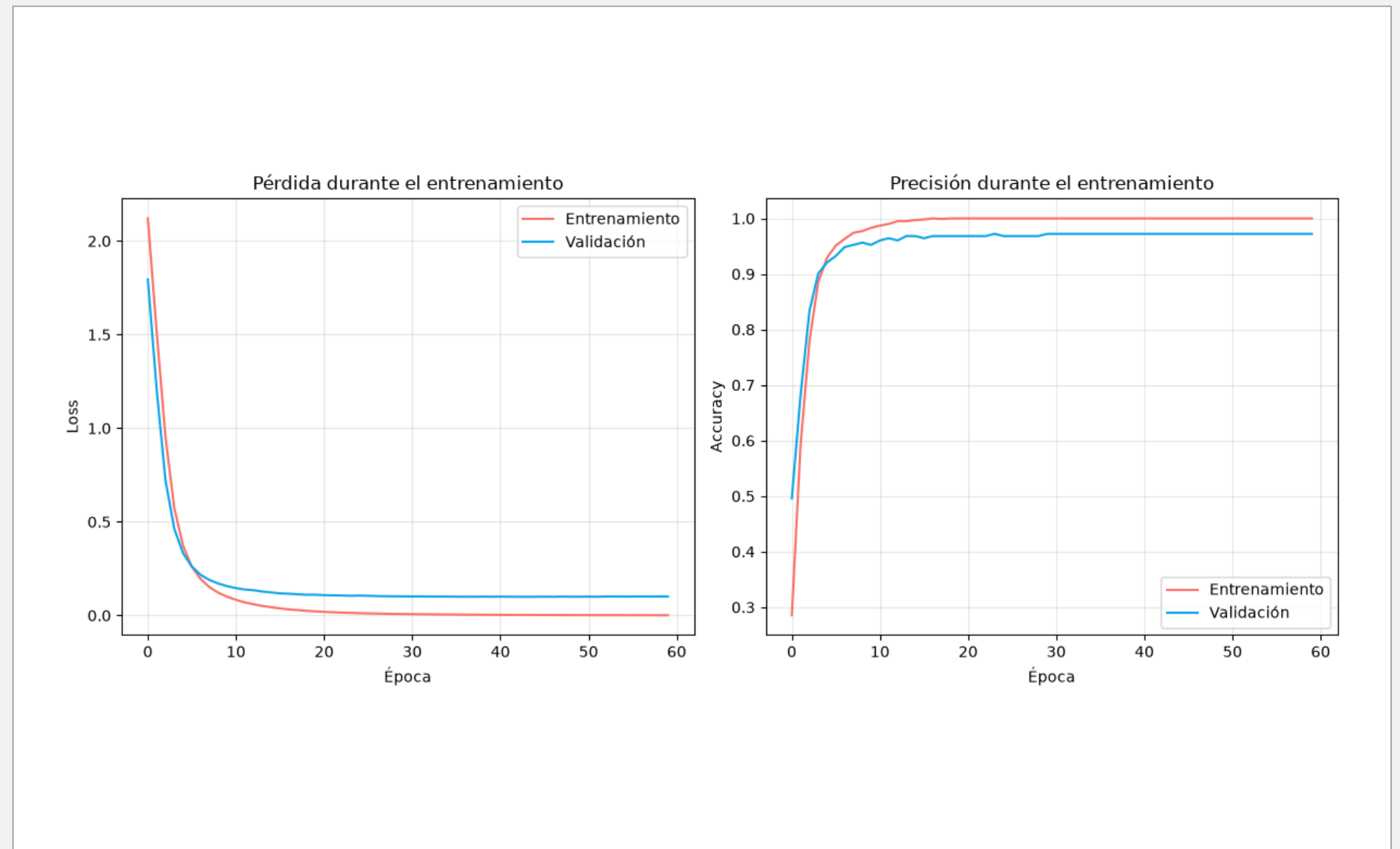


Entrenamiento: pérdida y descenso de gradiente

- La función de pérdida mide qué tan lejos están las predicciones de los valores reales
- El descenso de gradiente ajusta los pesos en la dirección que más reduce esa pérdida — el mismo algoritmo de la Unidad 3, aplicado ahora a miles de parámetros
- Backpropagation es el método eficiente para calcular el gradiente de la pérdida respecto a cada peso de la red, capa por capa

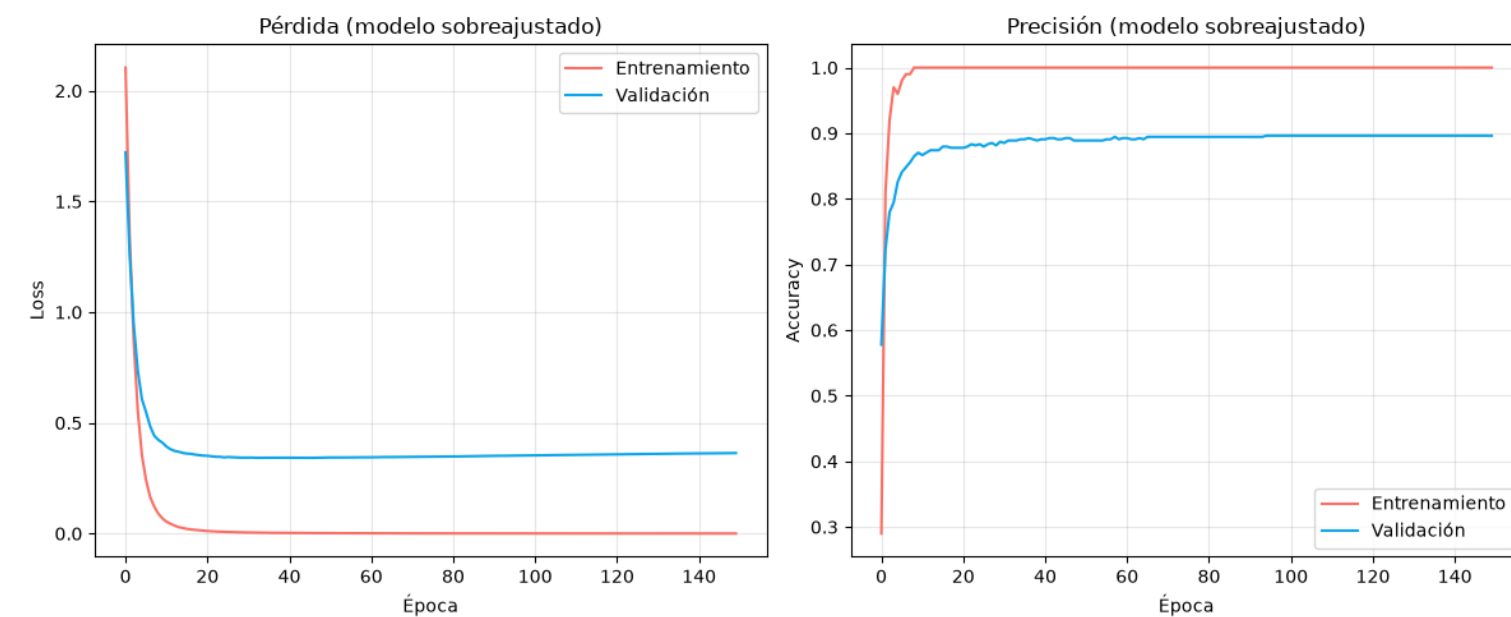
Ejemplo real: clasificación de dígitos (Digits)

- 1797 imágenes 8×8 de dígitos escritos a mano (10 clases) — más exigente que Iris para evaluar una red real
- Resultado real: 97.8% de exactitud en el conjunto de prueba

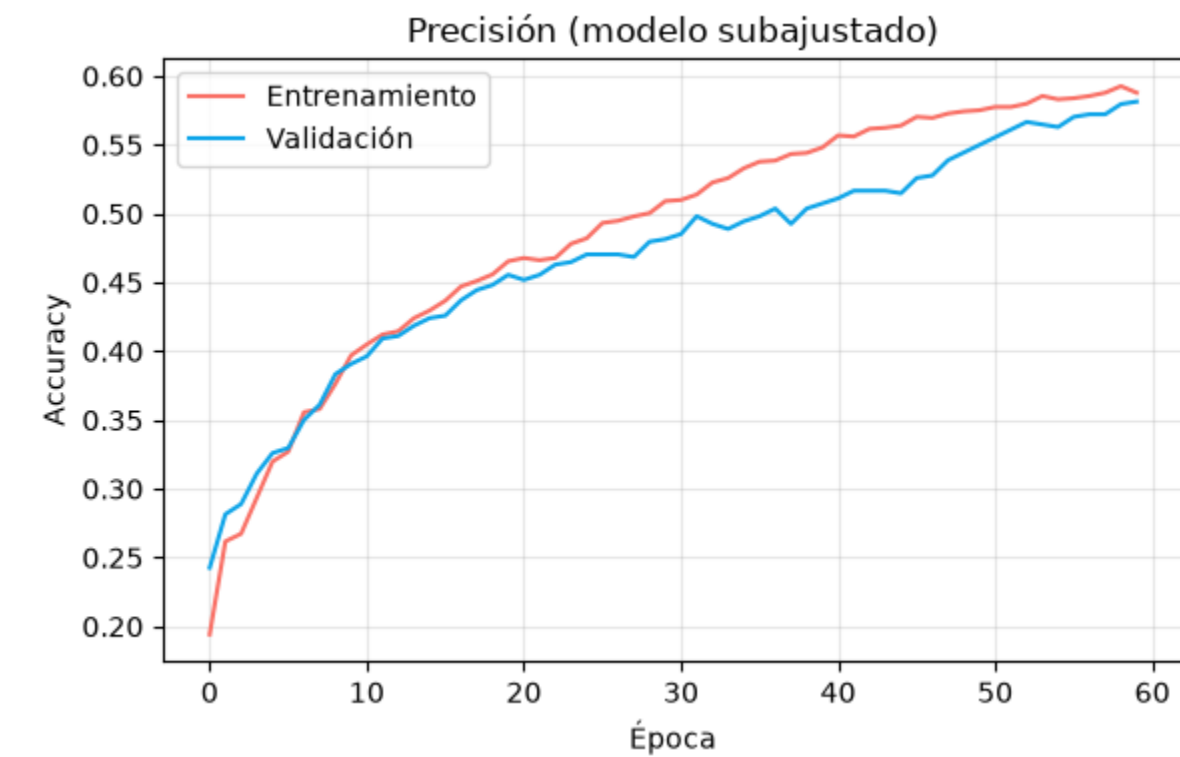


Sobreajuste y subajuste, con una red real

Sobreajustado (red grande, poca data)



Subajustado (red demasiado pequeña)



02

**Visión por
Computador (Clásica)**

Una imagen es una matriz de números

- Una imagen a color es un arreglo de alto $\times$ ancho $\times$ 3 canales (rojo, verde, azul)
- En escala de grises es un solo canal: cada píxel es un número entre 0 (negro) y 255 (blanco)
- OpenCV carga las imágenes en orden BGR, no RGB — una fuente común de errores si no se convierte antes de mostrarlas



Detección de bordes: Sobel

- Sobel calcula el gradiente de intensidad en X e Y con un kernel 3×3 — donde la intensidad cambia bruscamente, hay un borde
- La magnitud del gradiente combina ambas direcciones en un solo mapa de bordes



Binarización: umbral fijo vs. umbral de Otsu

- Un umbral fijo (127) no se adapta a cada imagen; el método de Otsu calcula el umbral óptimo (97 en este caso) a partir del histograma de la imagen
- Otsu busca el punto que mejor separa dos grupos de píxeles (fondo y objeto)

Umbral fijo = 127



Umbral de Otsu = 97

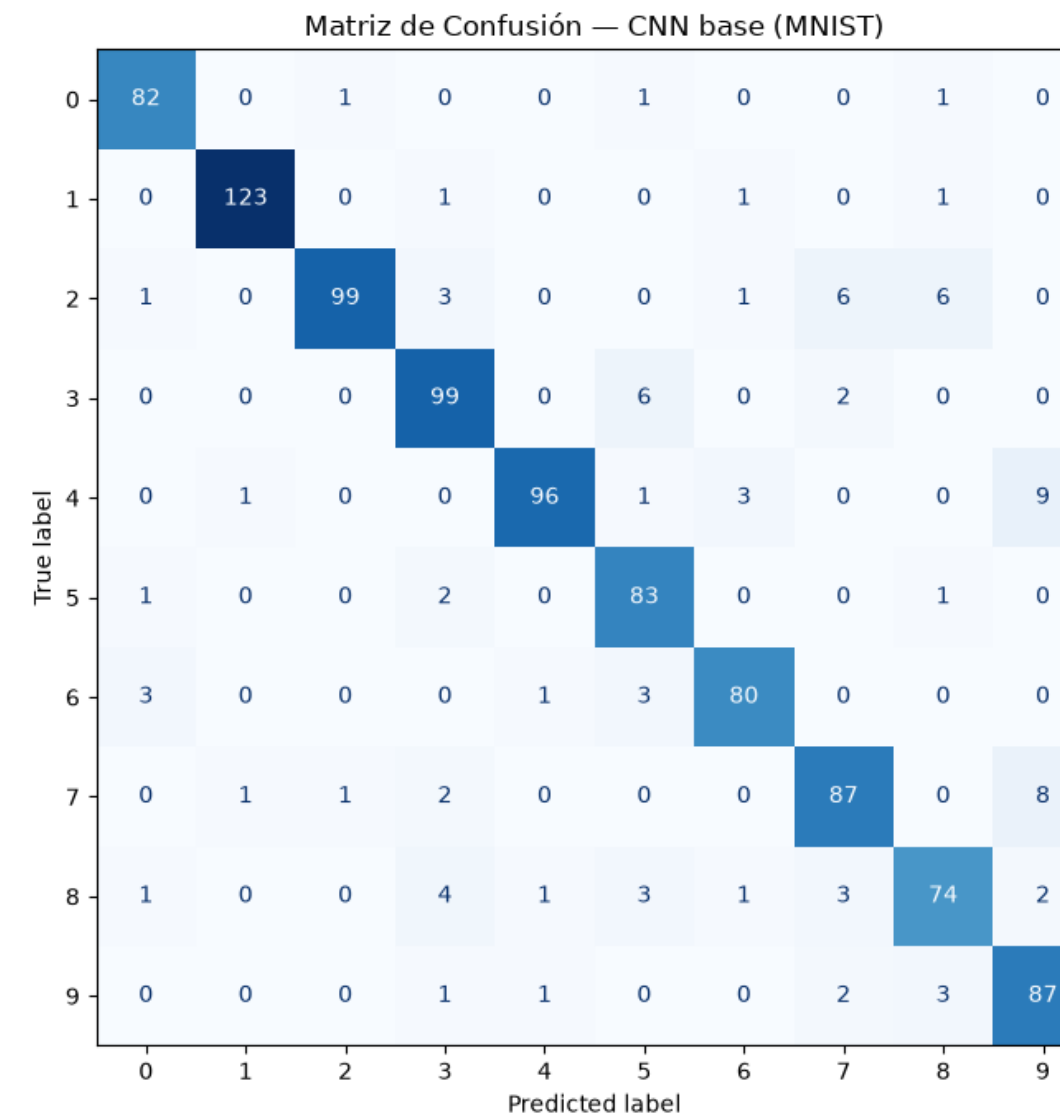


03

Redes Neuronales Convolucionales (CNN)

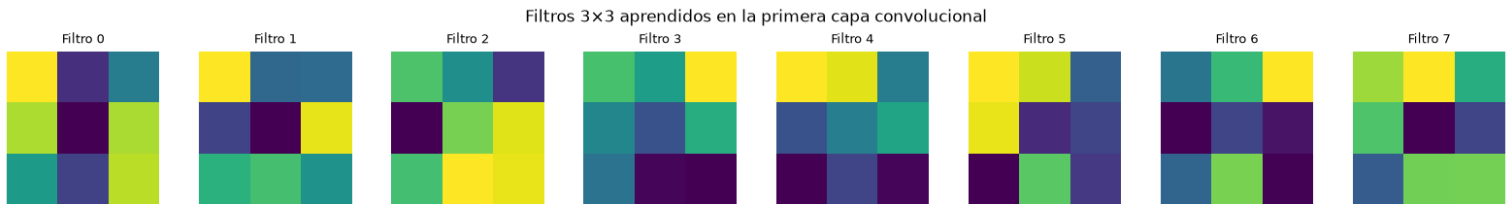
CNN base sobre MNIST: matriz de confusión real

Confusiones reales más frecuentes: $4 \rightarrow 9$ y $2 \rightarrow 7/8$ — dígitos que también se parecen a simple vista.



Qué aprende cada filtro convolucional

Filtros 3×3 aprendidos (primera capa)

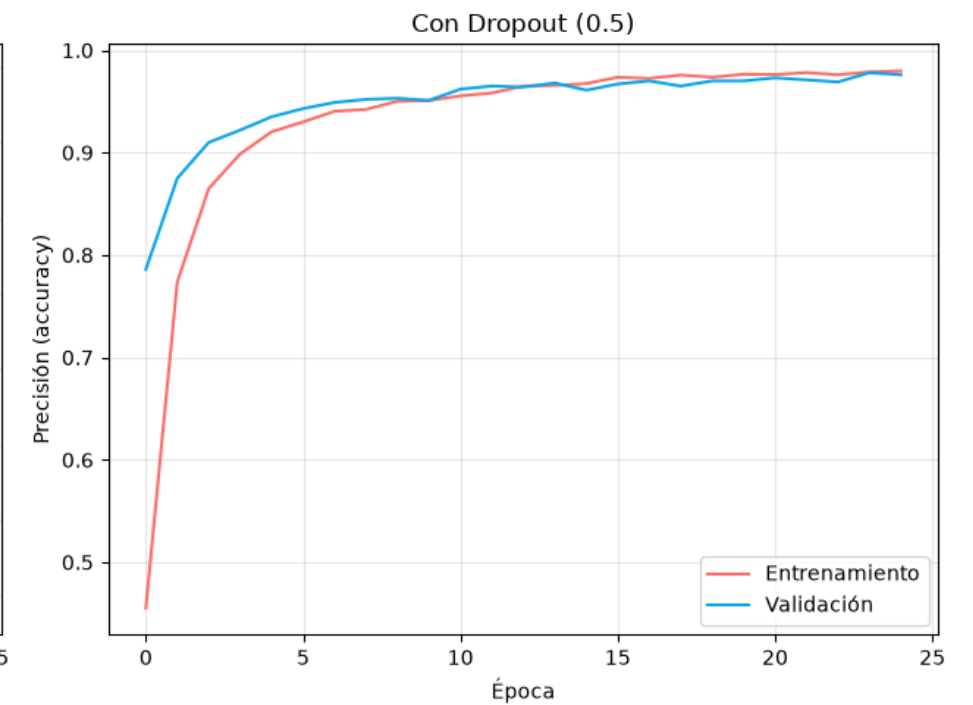
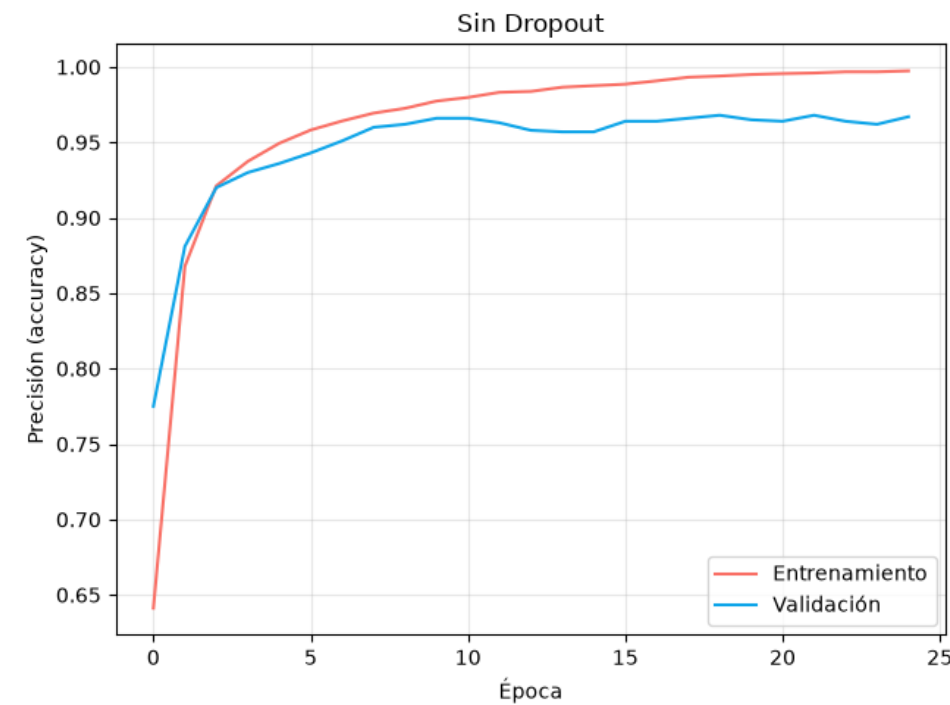


Mapas de activación sobre un dígito real



Dropout reduce la brecha entre entrenamiento y validación

Sin dropout, la brecha entre entrenamiento y validación se mantiene; con dropout (0.5), ambas curvas terminan mucho más cerca.



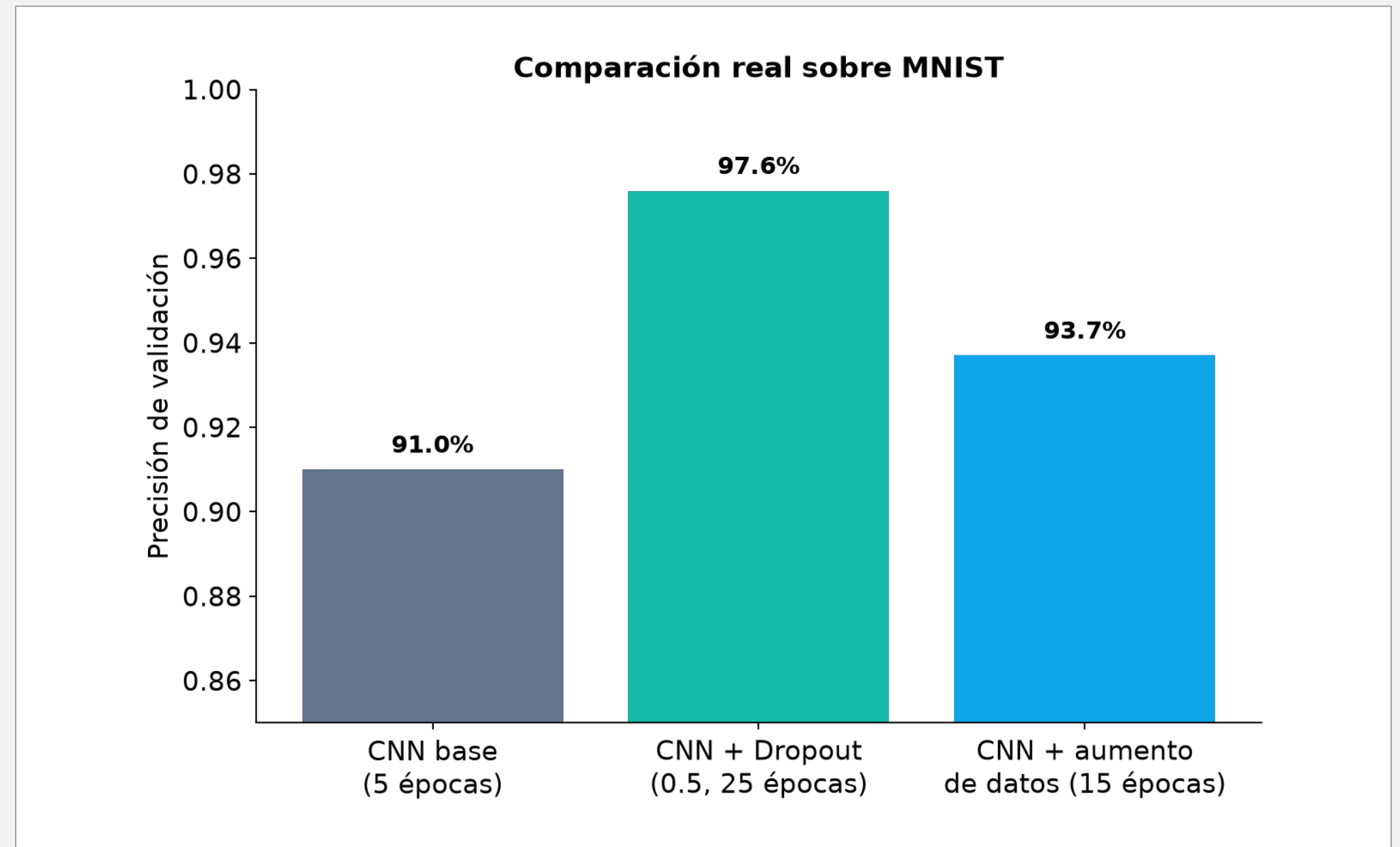
Aumento de datos: más variedad, sin más imágenes

- Pequeñas rotaciones, desplazamientos y zooms generan variantes realistas de la misma imagen de entrenamiento
- El modelo ve efectivamente más ejemplos sin necesidad de recolectar más datos, reduciendo el sobreajuste



Comparación real, y transferencia de aprendizaje

- Dropout es, de las tres variantes, la que más mejora la validación en MNIST
- Transferencia de aprendizaje (MobileNetV2 congelado + clasificador nuevo) alcanza 88.9% en 3 clases de CIFAR-10
- Reutilizar una red preentrenada es mucho más rápido que entrenar una CNN desde cero

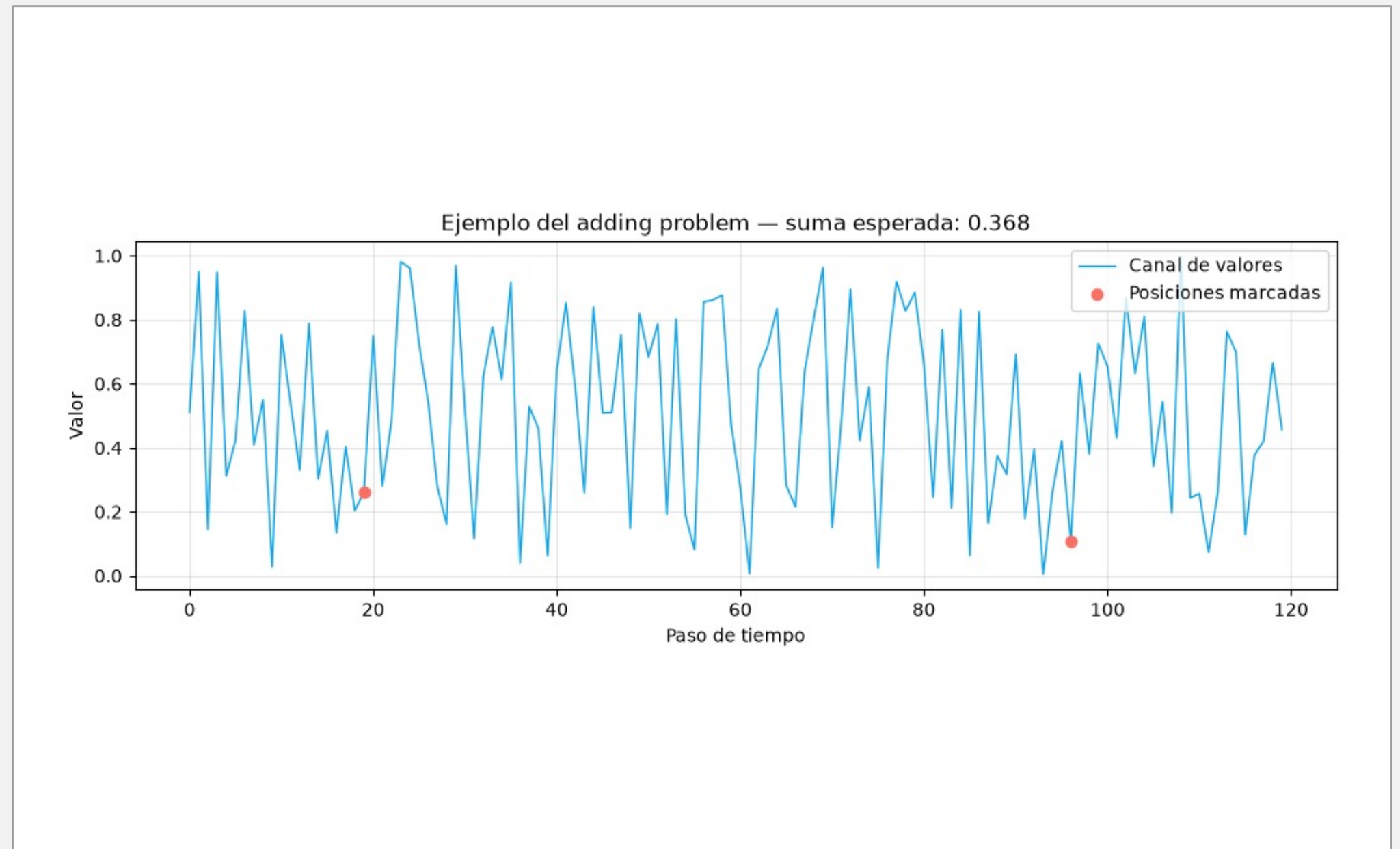


04

**Modelos
Secuenciales: RNN,
LSTM y GRU**

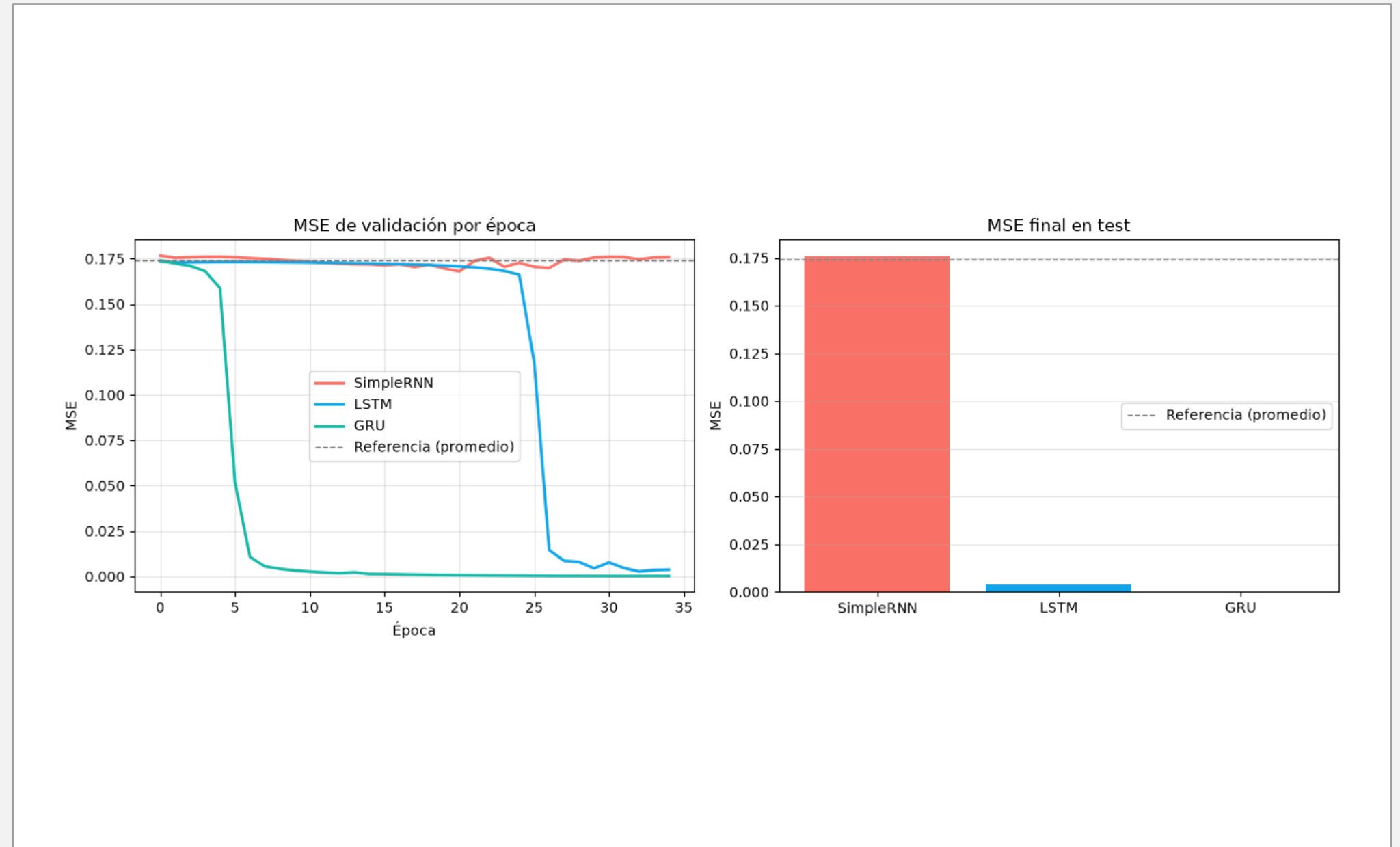
El "adding problem": dependencias largas

- Tarea sintética clásica: sumar los dos valores marcados, sin importar qué tan lejos estén entre sí en la secuencia
- Obliga al modelo a recordar información durante muchos pasos de tiempo — el punto débil de una RNN simple



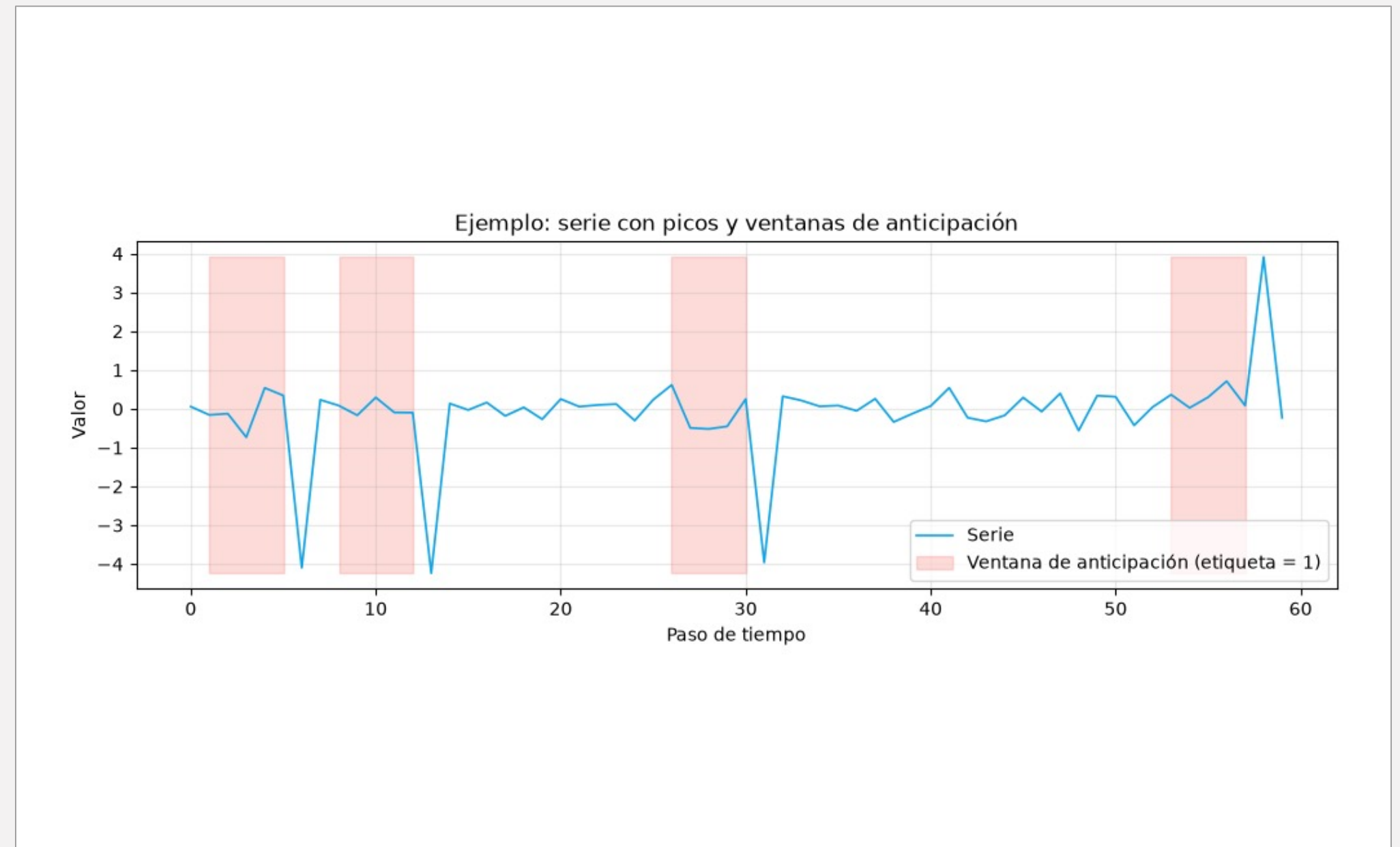
SimpleRNN no aprende; LSTM y GRU sí

- SimpleRNN se queda estancado en el error de referencia (predecir siempre el promedio) — no logra aprender la dependencia larga
- LSTM y GRU sí aprenden, gracias a sus "puertas" que controlan qué información recordar u olvidar



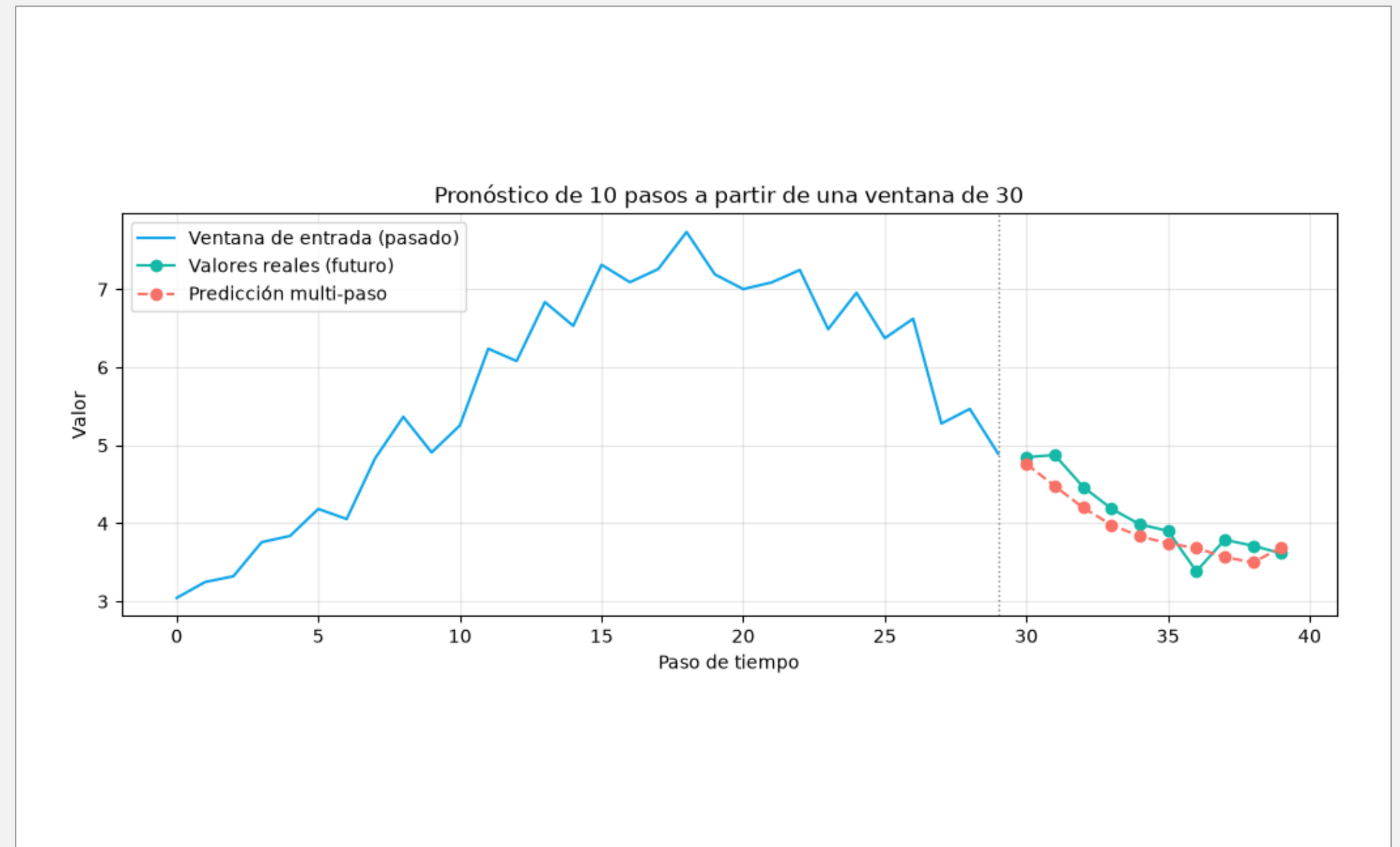
Bidirectional LSTM: cuando el futuro también importa

- Tarea: anticipar un pico antes de que ocurra — requiere ver el contexto que viene después del punto actual
- Una LSTM unidireccional casi no lo logra ($\text{recall} \approx 0.15$); una Bidirectional LSTM, que procesa la secuencia en ambos sentidos, lo detecta casi siempre ($\text{recall} \approx 1.00$)



Pronóstico multi-paso, sobre datos reales

A partir de una ventana de 30 pasos pasados, el modelo predice 10 pasos futuros de una sola vez.



05

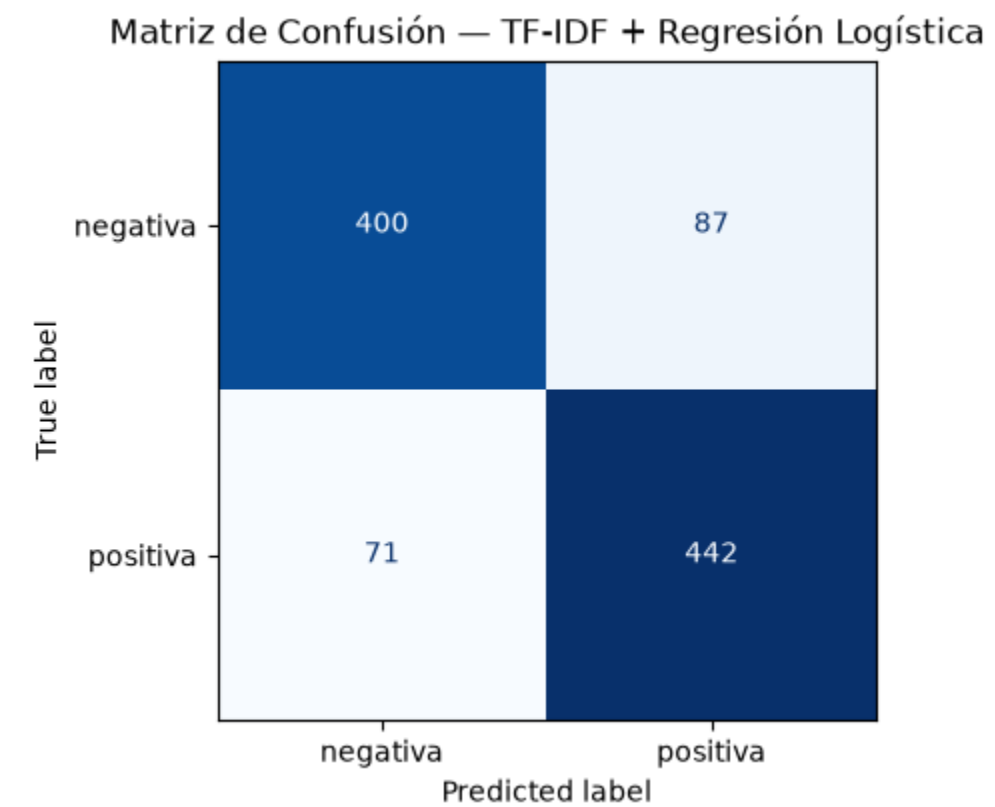
Procesamiento de Lenguaje Natural (PLN)

De palabras a números

- Los modelos no entienden texto directamente — hay que convertirlo en números
- TF-IDF: cada documento es un vector de frecuencias de palabras, ponderadas por qué tan distintiva es cada una
- Embeddings: cada palabra es un vector denso aprendido, donde palabras de significado similar quedan cerca entre sí
- Dataset de trabajo: reseñas de películas de IMDB, etiquetadas como positivas o negativas

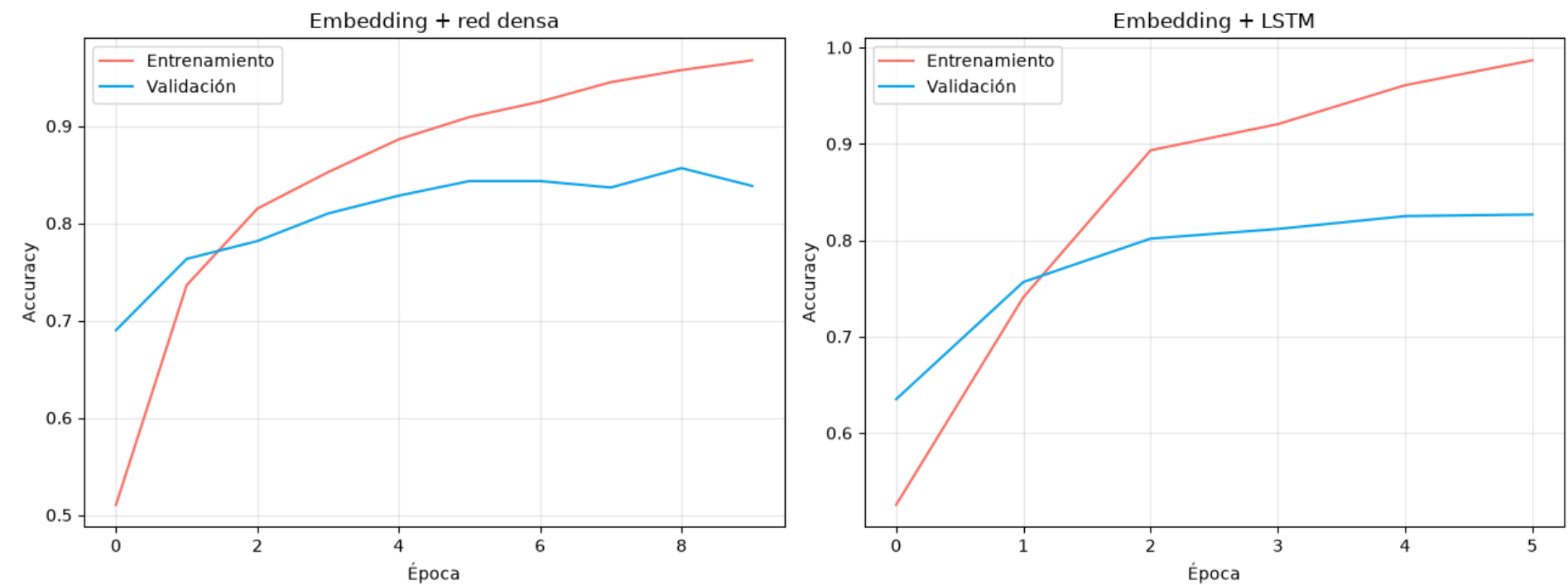
TF-IDF + Regresión Logística: matriz de confusión real

TN=400, FP=87, FN=71, TP=442 — un enfoque clásico, simple y ya bastante competitivo.



Embedding + red densa vs. Embedding + LSTM

- Ambos modelos muestran una brecha clara entre entrenamiento y validación: sobreajuste real, con relativamente pocos datos de entrenamiento
- La LSTM no supera aquí al enfoque más simple — más complejidad no siempre gana

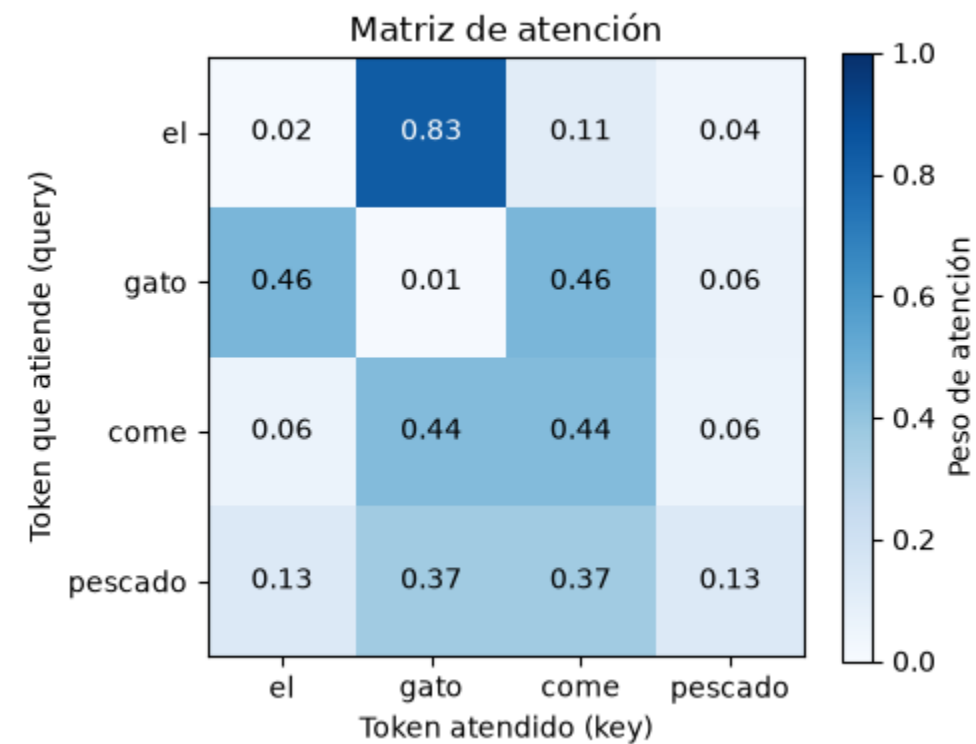


06

Transformers

Self-attention: cada palabra "mira" a las demás

Calculado a mano con NumPy: "gato" atiende fuertemente a "el" — el sujeto de la oración.

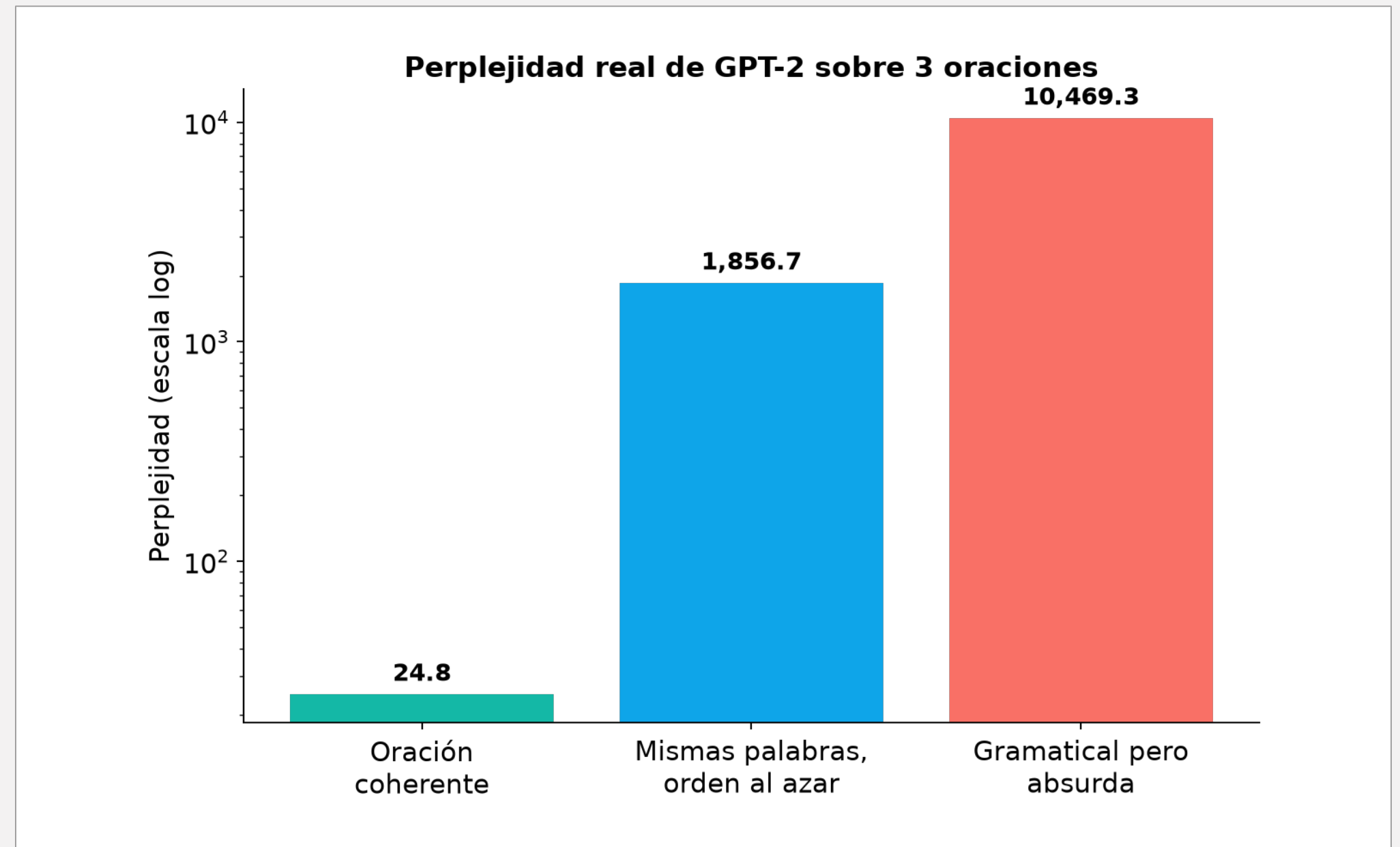


De RNN a Transformers

- Las RNN/LSTM procesan una secuencia paso a paso — no se pueden paralelizar, y aun así les cuesta recordar dependencias muy largas
- Self-attention calcula, en paralelo, qué tanto debe "atender" cada palabra a cada otra palabra de la secuencia
- Esta es la pieza central detrás de los modelos de lenguaje modernos (BERT, GPT, y los pipelines de Hugging Face)

Evaluar un modelo generativo: perplejidad

- Perplejidad = qué tan "sorprendido" está el modelo al leer un texto — menor es mejor
- GPT-2 (real): 24.8 en una oración coherente, pero miles de veces mayor en una versión con las mismas palabras desordenadas
- Útil para modelos generativos, donde no existe una única "respuesta correcta" que comparar como en clasificación



Resumen de la Unidad 4

Tema	Notebook del curso
Fundamentos de redes neuronales	1-fundamentos-redes-neuronales.ipynb
Visión por computador clásica	2-vision-por-computador.ipynb
Redes convolucionales (CNN)	3-redes-convolucionales.ipynb
Modelos secuenciales (RNN/LSTM/GRU)	4-modelos-secuenciales.ipynb
Procesamiento de lenguaje natural	5-procesamiento-lenguaje-natural.ipynb
Transformers	6-transformers.ipynb

¡Gracias!

De la neurona artificial a los transformers — la base de todo lo que viene en la Unidad 5 (agentes de IA).